

EduGenie

Google Gemini Powered Learning Assistant

Phase 2: Requirement Analysis

Field	Details
Project Title	EduGenie: Google Gemini Powered Learning Assistant
Phase	Phase 2: Requirement Analysis
Prepared by	Santhiya.N
Mentor	Durga
College / Team ID	Government Arts College Salem-7 (Autonomous))
Date	28/09/2026

1. Objective

This document lists everything EduGenie needs in order to be built and run: what the system must do (functional requirements), how well it must do it (non-functional requirements), and the hardware, software and services required.

2. Functional Requirements

ID	Requirement	Priority
FR1	The user can type a question and receive an AI-generated answer	High
FR2	The user can paste text and receive a short bullet-point summary	High
FR3	The user can enter a topic and number of questions and receive multiple-choice questions	High
FR4	The user can select options and submit to see correct/wrong answers and a score	Medium
FR5	The system shows a clear error message when the AI service fails	Medium
FR6	The interface provides tabs to switch between Q&A, Summarize and Quiz	Medium

3. Non-Functional Requirements

ID	Category	Requirement
NFR1	Performance	Responses appear within a few seconds for typical prompts (depends on the API)

ID	Category	Requirement
NFR2	Usability	Simple single-page interface, no login or registration
NFR3	Reliability	API failures are caught and shown as readable messages instead of crashing
NFR4	Security	The API key is stored as an environment variable and is not written in the code
NFR5	Portability	Runs on any system that has Python and a web browser
NFR6	Maintainability	Backend and frontend are kept in separate folders with clear modules

4. Hardware Requirements

Component	Requirement
Processor	Intel i3 or equivalent, or higher
RAM	4 GB minimum (8 GB recommended)
Storage	About 500 MB free space (for Python and libraries)
Network	Active internet connection (the Gemini API is cloud-based)

5. Software Requirements

Item	Details
Operating System	Windows 10 / 11
Language	Python 3.10 or higher (Python 3.14 was used)
Web Browser	Google Chrome
Tools	Command Prompt to run the server; Notepad to edit files
Python Libraries	fastapi, uvicorn, google-generativeai, pydantic
Cloud Service	Google AI Studio (to create a free Gemini API key)

6. Technology Stack

Layer	Technology
Frontend	HTML, CSS, JavaScript (Fetch API)
Backend	Python, FastAPI, Uvicorn (ASGI server)
AI Model	Google Gemini (Flash) via the google-generativeai library
Data Format	JSON for requests and responses

7. User Stories

As a...	I want to...	So that...
Student	ask a question in simple words	I can understand a concept quickly
Student	paste a long paragraph and get a summary	I can revise in less time
Student	generate a quiz on a topic	I can test my understanding
Student	see which answers were right or wrong	I know what to study again

8. Constraints and Assumptions

- The free tier of the Gemini API has request limits per minute and per day; heavy testing can trigger a quota error (HTTP 429).
- An internet connection is required for every AI request.
- Gemini model names can change over time; an alias name (gemini-flash-latest) is used to avoid breakage.
- The google-generativeai library shows a deprecation warning; moving to the newer google-genai library is planned as future work.
- The API key must be set again in Command Prompt each time a new window is opened.
- The application is tested with English input.